BuildCore Infrastructure: A Project-First, Database-Driven System for Construction Resource Management

Ishan Awasthi
Reg. No.: 24BAI0207
BCSE302L-Database Management Systems
Course Project Report

***Abstract*— Large construction companies run many concurrent projects, each consuming workers, machinery, materials and money drawn from shared pools. When this information lives in spreadsheets, resources are double-booked, costs drift apart and statuses go stale. This paper presents BuildCore Infrastructure, a web-based management console for senior managers and administrative staff of large builders, implemented with Flask and PostgreSQL. The system is organised around one principle: the project is the root of both the data model and the navigation. Every task, worker assignment, equipment deployment, material allocation, purchase and expense is bound to a project through foreign keys, and each project exposes a workspace of seven tabs. The relational schema contains fifteen tables, junction tables for many-to-many relationships, CHECK and UNIQUE constraints, and five SQL views that compute worker availability, equipment usage, duration-weighted progress, purchase totals and a unified financial ledger. Snapshot-locked rates, schedule-collision detection and read-only completed projects preserve auditability. The paper details the design, constraints, workflow and strengths of the system and discusses its limitations.**

***Index Terms*— construction management, relational database, PostgreSQL, SQL views, data integrity, resource scheduling, Flask.**

I. INTRODUCTION

Construction is among the most resource-intensive industries. A single builder may run dozens of concurrent projects, each consuming people, heavy machinery, materials and money drawn from shared company pools [8]. Decisions such as where a crane goes, which crew is free next week or whether a project is still within budget depend on information spread across site registers, purchase files and spreadsheets. When these sources disagree, the consequences are double-booked resources, unnoticed cost overruns and late decisions.

The relational model [1] offers a principled alternative: keep every fact once, relate facts through keys, and let the database enforce what is valid [3], [4]. This paper applies that idea to construction management and presents BuildCore Infrastructure, a web application built with Flask [6] and PostgreSQL [5]. BuildCore is aimed at senior managers and administrative staff of large builders, such as project directors, planning heads, procurement officers and finance controllers, who need a reliable portfolio-level view and a controlled way to commit resources rather than a tool for site-level data capture.

The main contributions are:

1) a project-first design in which the Projects tab is the root of both the data model and the user navigation;
2) a normalised schema of fifteen tables with foreign keys, junction tables, and CHECK and UNIQUE constraints that encode business rules;
3) five SQL views that compute worker availability, equipment usage, project progress, purchase totals and a unified financial ledger instead of storing them;
4) financial-integrity mechanisms: snapshot-locked rates, delivery-based cost recognition and read-only completed projects; and
5) a documented end-to-end workflow, from project creation to financial review, with screenshots and validation scenarios.

The remainder of the paper is organised as follows. Section II states the problem and objectives. Section III explains the target users and the project-first principle. Section IV describes the architecture, Section V the database design, Section VI the constraints and Section VII the SQL views. Section VIII walks through the application workflow, Section IX summarises the strengths, Section X reports validation, Section XI discusses limitations and future work, and Section XII concludes.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Companies that manage construction portfolios with disconnected tools typically face six recurring problems:

- workers are assigned to several tasks or projects with no dependable view of real availability;
- machines are double-booked, or their utilisation is untracked between tasks;
- material consumption is not tied to the project that used it;
- purchase orders and supplier records are not linked to project budgets;
- progress is estimated by hand and differs between reports; and
- status fields are edited manually, so they drift away from reality.

B. Objectives

1) Provide one unified system for projects, workers, equipment, materials, suppliers, purchases and expenses.
2) Model real operations with a normalised relational schema.
3) Enforce integrity with primary keys, foreign keys, CHECK and UNIQUE constraints and defined delete rules.

4) Represent many-to-many resource relationships accurately through junction tables.
5) Derive statuses, progress and cost totals dynamically with SQL views.
6) Give senior managers a project-first web interface for monitoring and decision making.
7) Protect the financial audit trail once resources are committed and projects are closed.

C. Scope

BuildCore covers planning-level and administrative management: project master data, task scheduling, resource assignment, procurement, cost tracking and reporting. Site-level activities such as attendance capture, drawing management and subcontractor billing are outside the present scope.

III. TARGET USERS AND THE PROJECT-FIRST PRINCIPLE

A. Target Users

The intended users are senior managers and administrative employees of large builders: project directors, regional heads, planning and contracts managers, procurement officers and finance controllers. They differ from site engineers in three ways. First, they think in projects: which projects are running, and how is each one doing? Second, they commit shared resources across several projects and therefore need to see conflicts before they occur. Third, they are accountable for budgets and need figures that can be audited. The interface therefore opens on an executive dashboard (Fig. 5) and treats the project as the unit of navigation.

B. Why the Project Is the Root

The project is the root of BuildCore in three senses.

Data root. Every operational record is reachable from exactly one project. Tasks, purchases, material allocations and expenses carry a `project_id` foreign key, while worker and equipment assignments reach their project through `project_tasks` (Table IV). Reporting on or removing a project therefore has an unambiguous scope.

Navigation root. Projects is the top entry of the sidebar. Opening a project displays a workspace of seven tabs: Overview, Tasks, Workers, Equipment, Materials, Purchases and Financials (Fig. 1). Resources are viewed and assigned only inside this workspace, so a manager always acts in the context of one project's dates, budget and tasks.

Lifecycle root. A project moves from planning through active work to completion. A completed project becomes read-only so that its financial history is preserved.

C. Registers and Workspace

Two kinds of pages exist. Company registers (Workers, Equipment, Materials, Task Types and Suppliers) are shared catalogues in which items are created and edited, and where a manager can see where each item currently is. The project workspace is where items are assigned. This separation prevents the common mistake of assigning a resource from a global list without regard to the project that needs it. Financials and Reports provide company-wide roll-ups.

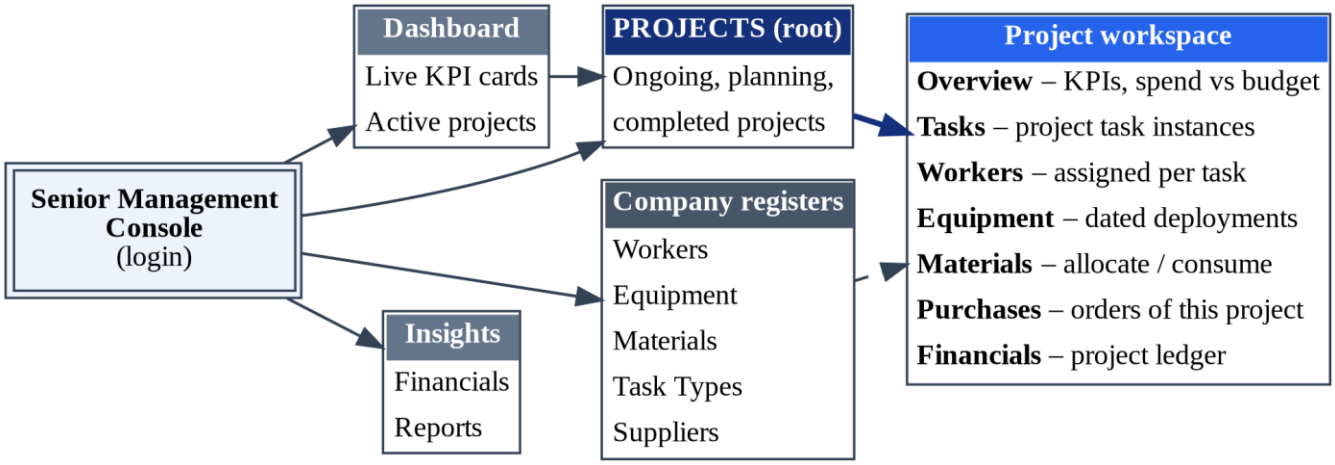


Fig. 1. Project-first navigation. Projects is the root; assignment of resources happens only inside the project workspace.

D. Executive Dashboard

The dashboard presents four live indicator cards (total projects with active and completed counts, workforce and fleet size, total purchases and total expenses) and a table of active projects showing type, location, budget and duration-weighted progress. All values are computed from SQL aggregates at request time; none is cached or hard-coded.

IV. SYSTEM ARCHITECTURE

A. Technology Stack

BuildCore is a server-rendered web application. Table I lists the technologies and their roles.

TABLE I
TECHNOLOGY STACK

Layer	Technology	Role
Presentation	HTML, Jinja2, Bootstrap 5.3	Pages, forms, tabs, dialogs
Application	Python, Flask 3.0 [6]	Routing, validation, sessions
Authentication	Flask-Login 0.6, Werkzeug	Login sessions, password hashing
Data access	SQLAlchemy 3.1 [7], psycopg2	Explicit SQL through text()
Database	PostgreSQL [5]	Tables, views, constraints
Configuration	python-dotenv	Connection settings, secret key

B. Request Flow and Modules

A request passes from the browser to a Flask route, which executes explicit SQL through SQLAlchemy [7], receives rows from a table or view, and renders a Jinja2 template (Fig. 2). SQLAlchemy is used mainly as a connection and transaction interface while the SQL itself is written by hand, so every database operation remains visible in the code. Ten blueprint modules group the routes by domain (Table II).

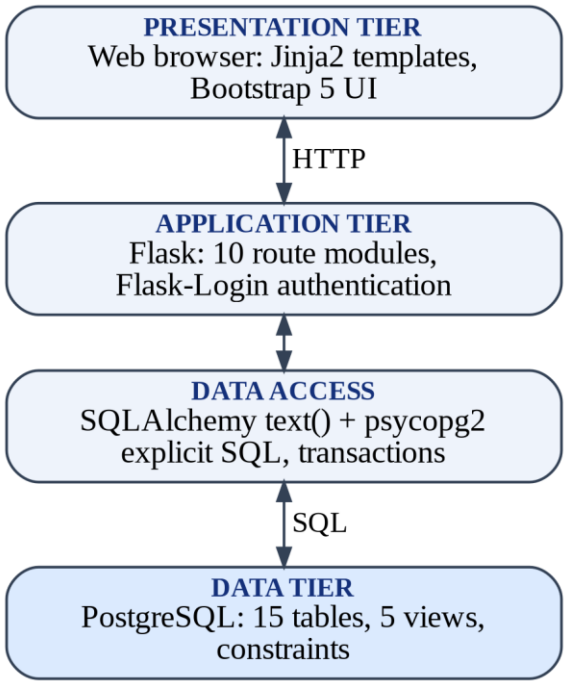


Fig. 2. Layered architecture of BuildCore.

TABLE II
ROUTE MODULES

Module	Responsibility
dashboard	KPI cards and active-project list
projects	Project records and workspace tabs
tasks	Task types and project task instances
workers	Worker register and task assignments
equipment	Equipment register and dated assignments
materials	Material catalogue and project allocation
suppliers	Supplier register
purchases	Purchase orders and line items
financials	Project ledgers and miscellaneous expenses
reports	Progress, financial, resource and procurement reports

C. Placement of Business Logic

BuildCore places business rules in the database wherever SQL can express them: keys and CHECK constraints reject illegal rows, and views compute derived values once for every screen. The application layer adds checks that need immediate user feedback before saving, such as date-overlap warnings, but it is not the only line of defence [3].

V. DATABASE DESIGN

A. Entities

The schema contains fifteen tables (Table III). Five are registers or catalogues, one stores credentials, and the remainder record project-owned facts.

TABLE III
TABLES AND THEIR ROLES

Table	Role
users	Login accounts (unique username, password hash)
projects	Master record: name, type, client, location, dates, budget, status
workers	Register: role, skill, daily rate, status
equipment	Register: type, hourly rate, status
materials	Catalogue: unit and unit cost
suppliers	Vendor register with contact details
task_types	Reusable activity templates (unique name)
project_tasks	Task instance of a project: dates, status, completion
task_workers	Workers on a task: hours, locked daily rate
equipment_assignments	Equipment on a task: dates, hours, locked hourly rate
material_usage	Quantity allocated and used per project and material
purchases	Order header: supplier, project, date, status
purchase_items	Order lines: material, quantity, unit price
expenses	Miscellaneous project costs
project_workers	Project-level staffing history (legacy)

B. Relationships

All relationships are one-to-many and are implemented as foreign keys. Table IV lists each parent, its child and the delete rule.

TABLE IV
FOREIGN KEYS AND DELETE RULES

Parent	Child	ON DELETE
projects	project_tasks	CASCADE
task_types	project_tasks	RESTRICT
project_tasks	task_workers	CASCADE
workers	task_workers	CASCADE
project_tasks	equipment_assignments	CASCADE
equipment	equipment_assignments	CASCADE
projects	material_usage	CASCADE
materials	material_usage	CASCADE
projects	purchases	CASCADE
suppliers	purchases	RESTRICT
purchases	purchase_items	CASCADE

Parent	Child	ON DELETE
materials	purchase_items	RESTRICT
projects	expenses	CASCADE
projects	project_workers	CASCADE
workers	project_workers	CASCADE

C. Many-to-Many Relationships

Construction data is naturally many-to-many, so five junction tables carry the operational model: `project_tasks` (projects and task types), `task_workers` (tasks and workers), `equipment_assignments` (tasks and equipment, over time), `material_usage` (projects and materials) and `purchase_items` (purchases and materials). A sixth table, `project_workers`, records project-level staffing and is retained for history, while day-to-day staffing is modelled at task level. Fig. 3 shows the resulting entity–relationship schema [2]; the standalone `users` table is omitted.

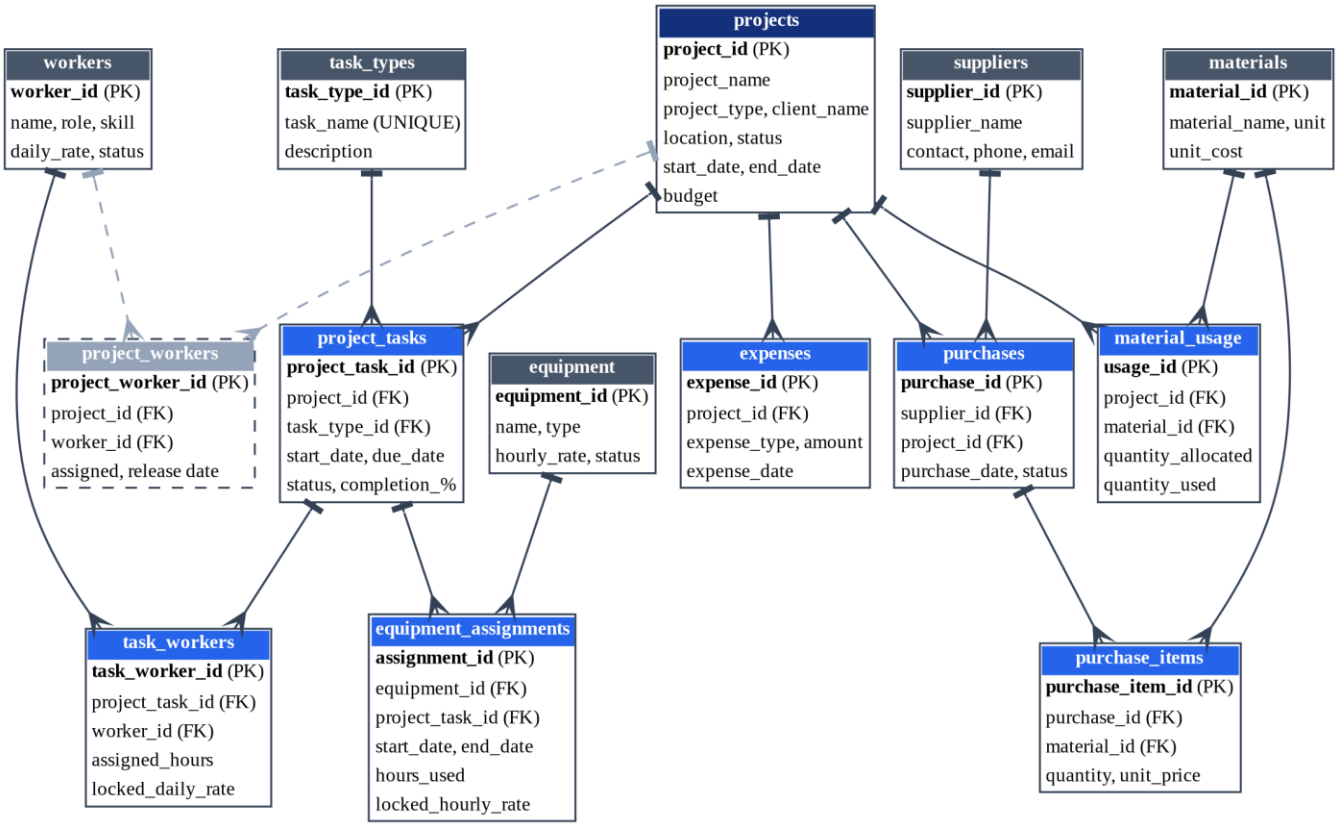


Fig. 3. Entity–relationship schema. Dark blue: root entity; grey: registers; blue: junction and transaction tables; dashed: legacy table. Crow's feet mark the many side. (Note: Standalone users authentication entity is documented in Table III).

D. Normalisation

Each fact is stored once. People, machines, materials, suppliers and task templates live in register tables; purchase orders are split into header and lines, which removes repeating groups; and derived quantities such as order totals, progress, availability and costs are computed in views rather than stored. The operational entities therefore satisfy third normal form [3], [4]. The deliberate exception is the pair `locked_daily_rate` and `locked_hourly_rate`, which copy the register rate at assignment time. This controlled denormalisation is required so that historical costs do not change when a rate changes.

E. Keys and Delete Rules

Surrogate serial keys identify every row, and monetary values use the NUMERIC type rather than floating point to avoid rounding errors. Project-owned data uses ON DELETE CASCADE so that no orphan rows remain, while reference data on which history depends (task types, suppliers, and materials referenced by purchase lines) uses ON DELETE RESTRICT. Because a cascade could silently erase an audit trail, the application adds a protection guard that blocks deletion of projects that already carry financial records.

VI. INTEGRITY CONSTRAINTS

BuildCore follows the principle that invalid data should be impossible rather than merely discouraged [4]. Table V lists the main declarative rules.

TABLE V
DECLARATIVE CONSTRAINTS

Constraint	Table(s)	Rule
budget >= 0	projects	Budget is never negative
check_end_date	projects	Expected end date is not before start date
daily_rate, hourly_rate, unit_cost >= 0	workers, equipment, materials	Rates and costs are non-negative
uq_project_task	project_tasks	A task type appears at most once per project
check_pt_due_date	project_tasks	Due date is not before start date
completion_percentage between 0 and 100	project_tasks	Valid percentage range
check_task_completion_status	project_tasks	Status is Completed if and only if completion is 100
check_task_not_started_percentage	project_tasks	A Not Started task has 0 percent completion
assigned_hours, hours_used >= 0	task_workers, equipment_assignments	Hours are non-negative
check_eq_end_date	equipment_assignments	End date is not before start date
check_tw_locked_daily_rate_non_neg	task_workers	Snapshot daily rate is non-negative
check_ea_locked_hourly_rate_non_neg	equipment_assignments	Snapshot hourly rate is non-negative
check_material_used	material_usage	Quantity used never exceeds quantity allocated
uq_project_material	material_usage	One line per project and material
quantity > 0, unit_price >= 0	purchase_items	Valid order line
amount >= 0	expenses	Costs are non-negative
check_release_date	project_workers	Release date is not before assigned date
UNIQUE username, UNIQUE task_name	users, task_types	No duplicate accounts or task templates

Two constraints deserve emphasis. `check_task_completion_status` makes task status and completion percentage a biconditional, so the two fields cannot disagree even if a script bypasses the web form. `uq_project_material` guarantees that stock allocated to one project cannot be merged with another project's stock.

When these constraints were introduced to an existing database, a migration first reconciled conflicting rows and then added the constraints inside a single transaction, after a database backup had been taken. Rules that depend on other rows, such as worker eligibility and date-overlap checks, are enforced in the application layer with immediate feedback (Section VIII).

VII. DYNAMIC SQL VIEWS

Instead of storing values that become wrong as soon as a related row changes, BuildCore computes them in five views (Table VI).

TABLE VI
SQL VIEWS

View	Computes
worker_status_view	Available or Assigned; On Leave and Inactive override
equipment_status_view	In Use if an assignment covers today; Maintenance and Retired override
project_progress_view	Duration-weighted progress and task counts
purchases_with_total_view	Order total from line items
expense_ledger_view	Unified ledger: materials, labour, equipment, other

Progress. Task durations differ, so a plain average would let a one-day task weigh as much as a three-month task. The progress view weights each task's completion percentage c by its duration d, measured in days from start date to due date inclusive:

$$P = (\sum c_i d_i) / (\sum d_i) \quad (1)$$

Availability. A worker is Assigned when at least one task the worker belongs to is Not Started or In Progress. An equipment unit is In Use when one of its assignment intervals contains the current date. Manual states such as On Leave or Maintenance take precedence.

Cost ledger. The ledger combines four cost categories. Labour converts assigned hours h to days at eight hours per day and applies the locked daily rate r; equipment multiplies hours used u by the locked hourly rate; material cost counts only purchases whose status is Delivered or Completed, which is accrual recognition:

$$C_{lab} = \sum (h_k / 8) \cdot r_{k,lock} \quad (2)$$

$$C_{eq} = \sum u_j \cdot r_{j,lock} \quad (3)$$

$$C_{mat} = \sum q_m \cdot p_m \quad (4)$$

Total recognised spend S, including miscellaneous expenses, and budget utilisation U for a project with budget B follow directly:

$$S = C_{mat} + C_{lab} + C_{eq} + C_{misc} \quad (5)$$

$$U = 100 \cdot S / B \quad (6)$$

VIII. APPLICATION WORKFLOW

Fig. 4 summarises the pipeline a manager follows and the database effect of each stage. The subsections below illustrate it with screenshots of the running system.

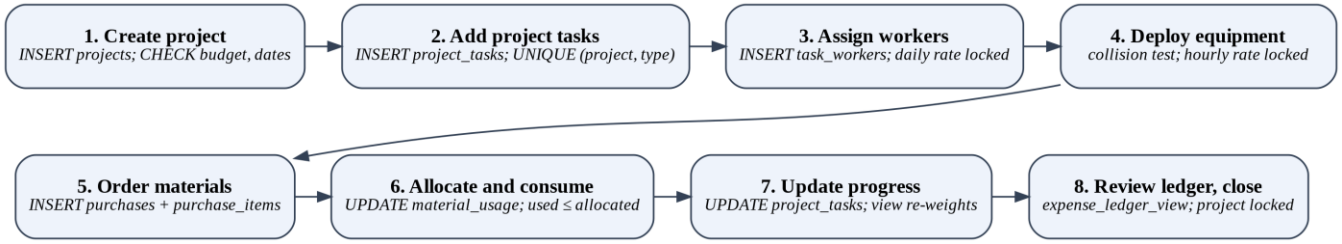


Fig. 4. Project pipeline from creation to financial review, with the database operation at each stage.

A. Login and Dashboard

Access requires authentication. After login, the dashboard (Fig. 5) shows portfolio indicators and the active projects with their progress. The View button opens the selected project's workspace. The sidebar separates Projects from the company registers, reflecting the project-first design.

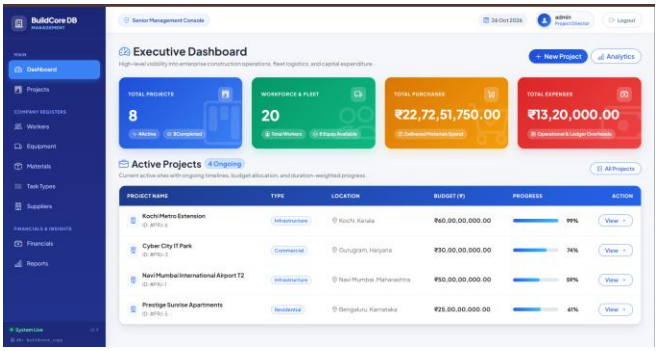


Fig. 5. Executive dashboard with live indicators and active projects.

B. Creating a Project

A manager creates a project by entering its name, type, client, location, start date, expected end date and budget. The database rejects a negative budget or an end date earlier than the start date. After saving, the application confirms the action and opens the new workspace with every counter at zero (Fig. 6).

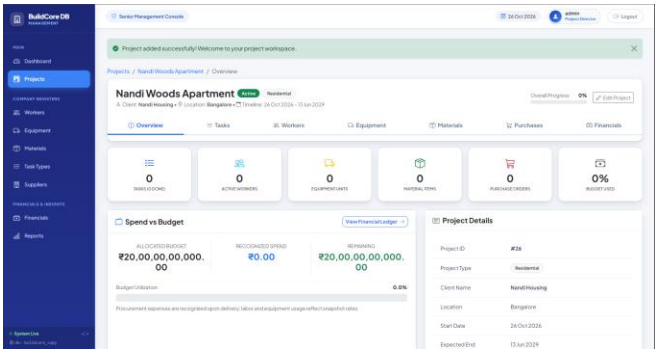


Fig. 6. Workspace of a newly created project: seven tabs, zero counters and its own budget panel.

C. Planning Tasks and Assigning Resources

In the Tasks tab, the manager adds task types from the reusable list and gives each a start date, due date and description; the pair of project and task type is unique. Workers and equipment are then assigned to a task of this project. In the equipment dialog (Fig. 7), choosing a project task limits the allowed dates to the task window, and the unit list shows each hourly rate and current state. If the unit is already booked, the dialog lists the existing booked periods and raises a Schedule Collision alert when the chosen dates overlap. Two periods A and B overlap when

$$s_A \le e_B \ \wedge \ e_A \ge s_B \tag{7}$$

and the submit action is blocked while a collision exists. On deployment, the current hourly rate is copied into `locked_hourly_rate`. Workers follow the same pattern with `locked_daily_rate`, and workers who are On Leave or Inactive cannot be assigned.

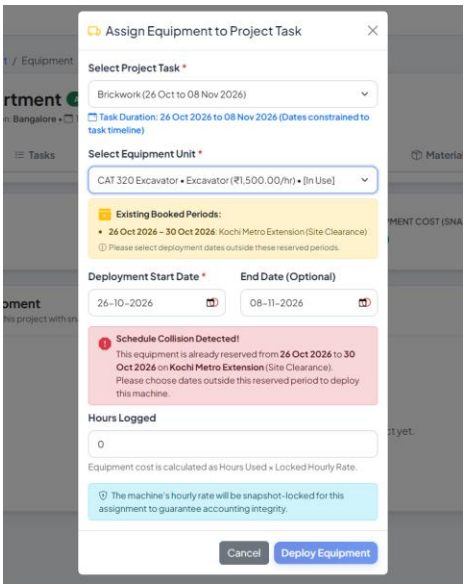


Fig. 7. Equipment assignment dialog showing booked periods and a schedule-collision alert.

D. Materials and Procurement

Materials are acquired through purchase orders. The manager chooses a supplier, creates a header row in `purchases` and adds one or more lines in `purchase_items`. An order's status moves through Pending, Shipped, Delivered or Cancelled, and only delivered orders enter the cost ledger. Materials are then allocated to the project in `material_usage` and consumption is recorded against the allocation; quantity used can never

exceed quantity allocated, and each project and material pair has exactly one line.

E. Progress, Financial Review and Closure

Editing a task to 100 percent sets it to Completed, and the constraint forbids any other combination. The Overview tab recomputes project progress from `project_progress_view`, and its Spend versus Budget panel reads `expense_ledger_view`. When all work is finished, the project is marked Completed and its workspace becomes read-only (Fig. 8), which locks task workflows and resource deployments for audit.

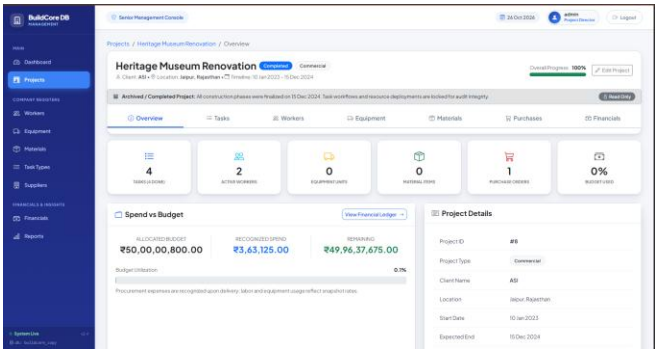


Fig. 8. A completed project: read-only banner, final progress and spend versus budget.

IX. KEY FEATURES AND STRENGTHS

1) **Project-first organisation.** One root for data and navigation matches how senior managers think and bounds every query and report.
2) **Database as source of truth.** Constraints and views apply to every client, including direct SQL, not just the web interface.
3) **Derived status.** Worker and equipment states come from assignment records, so stale "Assigned" flags cannot occur.
4) **Collision prevention.** Booked periods are shown and overlapping deployments are blocked before saving.
5) **Snapshot pricing.** Locked rates keep historical labour and equipment costs unchanged when register rates change.
6) **Material isolation and accrual costing.** One stock line per project and material, and spend recognised only on delivery.
7) **Audit protection.** Completed projects are read-only, restrictive delete rules protect reference data, and a guard protects projects with financial records.
8) **Executive visibility.** The dashboard, financial roll-ups and reports consolidate all projects.
9) **Reproducibility.** Schema, seed data, setup script, versioned migrations and backups allow the database to be rebuilt and evolved safely.

TABLE VII
SPREADSHEET PRACTICE VERSUS BUILDCORE

Aspect	Spreadsheet practice	BuildCore
Availability	Manual notes	Computed from assignments
Double-booking	Found late	Blocked at assignment
Progress	Hand estimate	Duration-weighted view
Rates	Overwritten, history lost	Locked per assignment
Costs	Scattered files	Unified ledger view
Status	Edited by hand	Derived from records
Audit	Easily altered	Read-only completed projects

X. VALIDATION

BuildCore was validated through scripted QA checks kept with the code and through side-by-side demonstrations in which each action in the browser was verified with SQL against the same database. Table VIII summarises the scenarios and the behaviour expected from the database.

TABLE VIII
VALIDATION SCENARIOS

Scenario	Action	Expected database behaviour
Live aggregates	Open dashboard	KPI values equal COUNT and SUM queries
Project isolation	Open workspace tabs	Tab counts equal rows filtered by project_id
Resource collision	Assign a booked worker or machine	Collision alert; submit blocked; status reads Assigned or In Use
Status sync	Set a task to 100 percent	Status becomes Completed; a direct UPDATE to 75 percent is rejected
Procurement	Mark an order Delivered	Material cost enters the ledger only after delivery
Snapshot pricing	Change a register rate	Existing assignment costs are unchanged
History protection	Delete a project with financial records	Deletion is blocked

XI. LIMITATIONS AND FUTURE WORK

Limitations. (i) The users table stores credentials only, so there is no role or permission model. (ii) Date-overlap prevention is implemented in the application layer; concurrent requests could in principle book the same unit, so a database-level guarantee would be stronger. (iii) Labour cost is computed from assigned hours at an eight-hour day rather than from recorded attendance. (iv) Materials are tracked as per-project allocations, not as warehouse stock across sites. (v) The `project_workers` table is not part of the current operational flow.

Future work. Role-based access for directors, planners, procurement staff and auditors; PostgreSQL exclusion constraints on assignment date ranges to enforce non-overlap inside the database; an audit-log table; timeline and resource-calendar views; a REST interface and notifications; and integration with attendance data.

XII. CONCLUSION

BuildCore shows that a construction management tool for senior managers becomes simpler and more trustworthy when the project is the root of both the data model and the navigation. Fifteen normalised tables, junction tables for many-to-many relationships, declarative constraints and five SQL views let the

database, rather than manual entry, decide what is valid and what is currently true. Snapshot-locked rates, delivery-based cost recognition and read-only completed projects keep the financial record auditable. The workflow from project creation to financial review can be demonstrated end to end, and the limitations identified above give a clear path for extending the system.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] P. P. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] The PostgreSQL Global Development Group, "PostgreSQL documentation." [Online]. Available: https://www.postgresql.org/docs/

[6] Pallets, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[7] SQLAlchemy, "SQLAlchemy documentation." [Online]. Available: https://docs.sqlalchemy.org/

[8] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA, USA: PMI, 2021.